(continued)

```
        }

        data = {
            "name": "some venue name",
            "description": "some venue description",
        }
        response = self.client.post("/api/v1/bands/venue/", data,
            content_type="application/json", headers=headers)
        # Asserts about the response go here
        ...
```

The payload data for the POST call

Using a JSON content type and the headers dictionary to post to the API endpoint

In the Ninja docs interface, every endpoint in your API gets displayed as a collapsible line. You can expand any endpoint to see more details about each call. In the expanded view, there is also a Try it Out button. This button is a little misleading; it should read Get Ready to Try it Out. Pressing it creates another button labeled Execute, which you press to actually do something. It is a two-step process for the scenarios where you need to provide data. For a GET with no arguments, it is an extra step. Try executing one of your GET endpoints to see the results. Ninja gives you a bunch of info: the `curl` call equivalent of what you just ran, which you can copy/paste into a terminal, the URL you just executed against, the response body, and the headers that came back with it.

If you attempt to run the POST for creating a `Venue`, you will get a Permission Denied response. This is because you didn't provide an authentication key. To do so, scroll to the top of the screen, and click the Authorize button. This pops open a new window, which in our case, shows the name of the HTTP header and an input box for the key. Fill it in, press Authorize, and all subsequent calls will include the authentication header. Note that this information isn't long-lived. If you exit your development server and come back, you may need to re-enter the key.

Once you've authenticated, expand the POST call for creating a venue, and then press the Try it Out button. A text field gets added, containing a template of the JSON payload needed for the call. Replace the placeholder values with actual content, and then press Execute. Like with the GET calls, you'll see the `curl` command, the URL used, and the response header and content.

If you scroll to the bottom of the docs interface, you'll see a listing of all your serialization schemas. Expand any of these to see the required format for the fields. The docs tool is very handy for testing and can be left on for developers working with your API. If you've secured everything with keys, this interface really is no different from the API itself. If you want to be a little more subtle about things, you can ask Ninja to use Django authentication before giving access to the interface or turn it off altogether. Details on how to do that can be found here: https://django-ninja.dev/guides/api-docs/.